

THERE’S MORE THAN ONE WAY: MODE COLLAPSE IN RLVR & MODEBENCH

Anonymous authors

Paper under double-blind review

ABSTRACT

A model can answer more reliably while producing a narrower range of successful outputs. Binary rewards in reinforcement learning with verifiable rewards (RLVR) treat repeated and different successes alike. We introduce MODEBENCH, five executable domains whose canonical keys identify verified alternatives, and ReplayMaxRL, which uses those same keys to organize memory. Fresh MaxRL updates learn from current samples; replay revisits one exemplar per retained key with equal weight, independent of rediscovery frequency. Longitudinal RLVR comparisons find increasing correct-key concentration in Graph and PantryPlan. Across three model scales, matched comparisons show that replay improves held-out correctness and sampled breadth under Dr.GRPO and complements MaxRL. At 3B, adding replay to MaxRL raises domain-averaged `pass@8` from .547 to .701 and `distinct@8` from .744 to 1.107. Harder constructions test replay beyond the original tasks; prompt and sampling interventions probe which alternatives remain accessible. Hosted evaluations find accurate outputs concentrated on few verified modes. An idealized analysis explains how gradient updates can amplify common successes and how memory can supply signal after fresh samples stop revisiting them. Together, these studies connect the measurement of verified alternatives to their continued use in learning, even after fresh sampling concentrates.

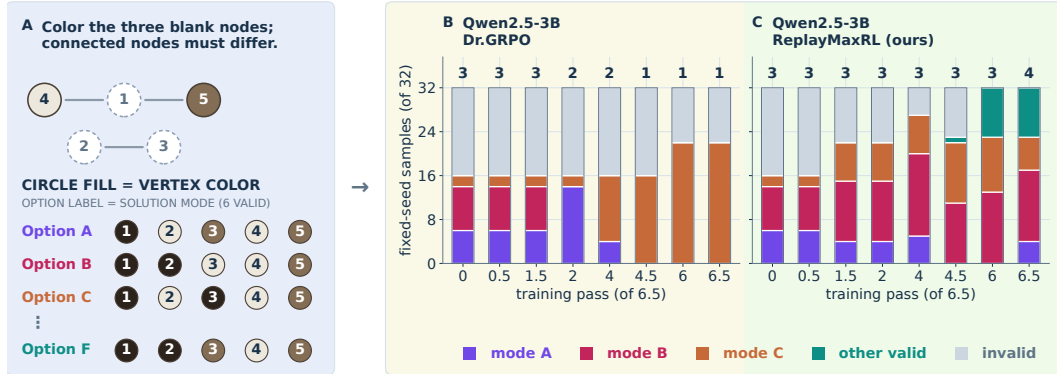


Figure 1: **Correctness can rise while sampled alternatives narrow.** One graph admits six valid colorings. Bars pool four eight-sample draws from matched Qwen2.5-3B runs. At pass 6.5, Dr.GRPO produces 22/32 correct outputs from one mode; ReplayMaxRL produces 32/32 across four. Circle fills mark vertex colors; dashed rings mark vertices left uncolored in the prompt. This illustrates the distinction; Figure 6 separates the effects of the fresh objective and replay.

1 INTRODUCTION

A language model can satisfy the same request in many ways: it can offer different explanations, generate different plans, or write different programs that meet the same requirements. These alternatives give users choices and may remain useful as preferences or constraints change. As a model becomes more successful, what happens to the diversity of successful outputs it can produce?

We study this question in reinforcement learning with verifiable rewards (RLVR), which trains a model using executable correctness feedback (Guo et al., 2025). Binary rewards treat a familiar correct solution and a different correct solution alike. Figure 1 illustrates why correctness and verified alternatives must be measured separately. Longitudinal saved-output comparisons find increasing correct-key concentration in Graph and PantryPlan under Dr.GRPO and GRPO at all three scales (Appendix H.1). The matched factorial separates replay from the different fresh objectives shown in the illustration.

MODEBENCH turns verified alternatives into a shared unit of measurement and memory. Each validator returns correctness and a canonical execution key: cosmetic variants can share a key, while different accepted computations can reach the same answer. ReplayMaxRL uses these same keys to retain one verified exemplar per admitted mode. Fresh MaxRL updates learn from current samples; equal-weight replay keeps stored successes involved even when they become rare in those samples. Repeated discovery does not earn a key more memory weight.

Our experiments ask whether this memory adds value beyond fresh learning, whether balancing keys matters, and how the results extend across models and task constructions. Prompt interventions, larger sampling budgets, and coarser keys test what the observed breadth represents. Matched comparisons find improved held-out success and sampled breadth, with reduced conditional concentration in the Graph/Pantry Dr.GRPO comparisons. Hosted evaluations test the measurement’s broader relevance without identifying a training cause. A Pantry outage study tests adaptation; improved correctness also contributes to its gains (Appendix L.1).

2 MODEBENCH: MEASURING SUCCESSFUL ALTERNATIVES

For a prompt x with specification s_x , a validator maps an output y to failure $\perp$ or a canonical key $V(y, s_x) \in \mathcal{C}_x$. Two accepted outputs count as the same mode exactly when their keys agree. Training sees only keys generated by the current policy; neither the full valid-key set $\mathcal{C}_x$ nor its size is supplied to the learner or replay bank.

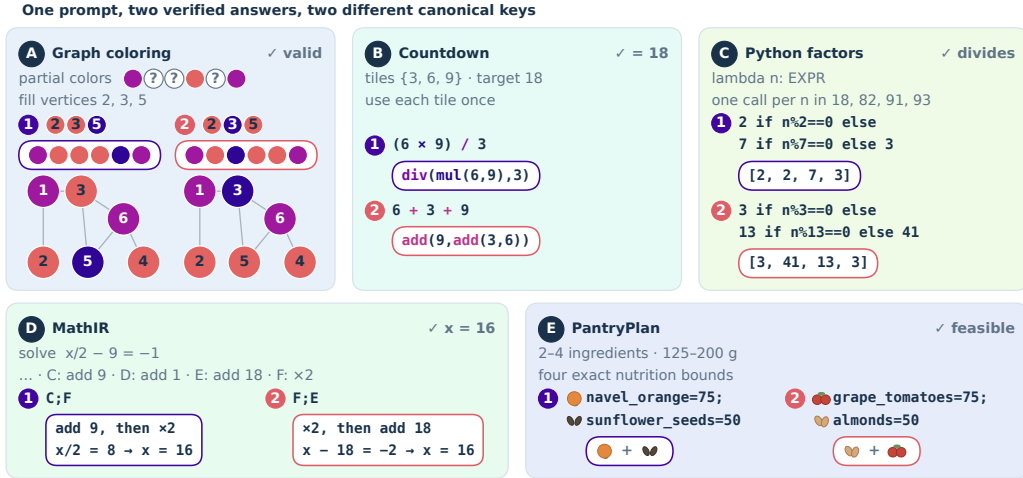


Figure 2: **One prompt can have different verified outputs.** Keys identify a color vector in Graph, a normalized computation tree in Countdown, an executed return vector in Python, an algebraic state trajectory in MathIR, or an ingredient support in PantryPlan. Cosmetic aliases merge; different accepted computations or configurations remain distinct under each domain’s execution rule.

Figure 2 shows the five domains. Countdown and MathIR capture different routes to an answer, Python captures different tested behaviors, and Graph and PantryPlan capture different valid configurations. This distinction matters when interpreting breadth: an additional valid configuration and an additional computation route are both verified modes, but they are not the same kind of alternative. The keys measure executed outputs, not latent reasoning strategies or the usefulness of a solution in a later task. Appendix D gives the exact key and alias rules, with verbatim prompts in Appendix E.

2.1 SUCCESS AND SAMPLED BREADTH

Let $P = \Pr[V(Y, s_x) \neq \perp]$ and, when $P > 0$, let $q_c = \Pr[V(Y, s_x) = c \mid V(Y, s_x) \neq \perp]$. For K independent samples, D_K counts their distinct correct keys. The promptwise identities are

$$\underbrace{\Pr[D_K > 0]}_{\text{pass@K}} = 1 - (1 - P)^K, \quad \underbrace{\mathbb{E}[D_K]}_{\text{distinct@K}} = \sum_c [1 - (1 - Pq_c)^K]. \quad (1)$$

The first endpoint measures success probability; the second measures an expected number of verified keys. We use $K = 8$ for the training endpoints. Their difference, *extra modes*, counts verified keys beyond the first success. Both raw breadth and extra modes can increase because correctness increases; neither by itself establishes a more balanced q .

Conditional collision measures that allocation directly:

$$C(q) = \sum_c q_c^2, \quad B_2 = 2P - P^2 C(q), \quad (2)$$

where $B_2 = \mathbb{E}[D_2]$. Thus C is the probability that two independent correct outputs share a key. Greater C means greater concentration. We call an observed increase in this conditional concentration *mode collapse*; finite samples do not establish literal extinction of unobserved modes. At fixed P , majorization toward a more concentrated q lowers expected breadth, but a change in C alone does not imply majorization or a change in total support (Appendix B).

2.2 HARDER TASKS WITH THE SAME NOTION OF A MODE

Levels retain each domain’s validator and canonical keys while changing problem structure. Level 1 is the original construction. Level 2 increases structural demands, such as hidden graph vertices and required Countdown operands, while preserving designated valid-mode-count histograms. Development checks confirm lower frozen Qwen2.5-0.5B success in every domain. Figure 3 measures frozen Qwen2.5-7B success and breadth on held-out Levels 1–3; Appendix Figure 9 includes all four model scales.

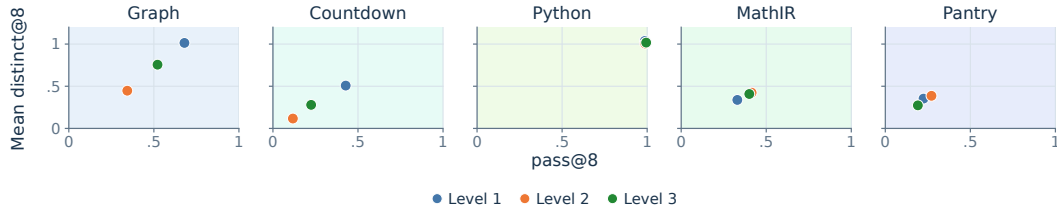


Figure 3: **Frozen Qwen2.5-7B across Levels 1–3.** Five domains show pass@8 versus mean distinct verified modes per eight draws; colors identify levels. Each point averages four independent eight-draw groups on 128 held-out prompts at $T = 1$. Appendix Figure 9 compares four model scales under the same evaluation protocol across all five domains and three levels.

A scale-calibrated extension targets the same fixed Qwen2.5-0.5B Level-1 success rates with larger frozen models. Only pass@1 and pass@8 are fitted; distinct@8 remains an independent measurement. These construction checks precede the replay comparison in Experiment 3. Native training evaluations use separate populations and parts of the protocol, so cross-level differences do not isolate difficulty (Appendix D.3).

3 REPLAYMAXRL: LEARN FROM NEW SAMPLES AND REMEMBER SUCCESSES

The execution keys used to measure alternatives also determine what the policy remembers. Figure 4 shows the two learning paths: binary rewards train on fresh samples, while replay revisits previously verified keys. Both use policy-generated responses; neither requires gold solutions or a precomputed catalogue of valid modes.

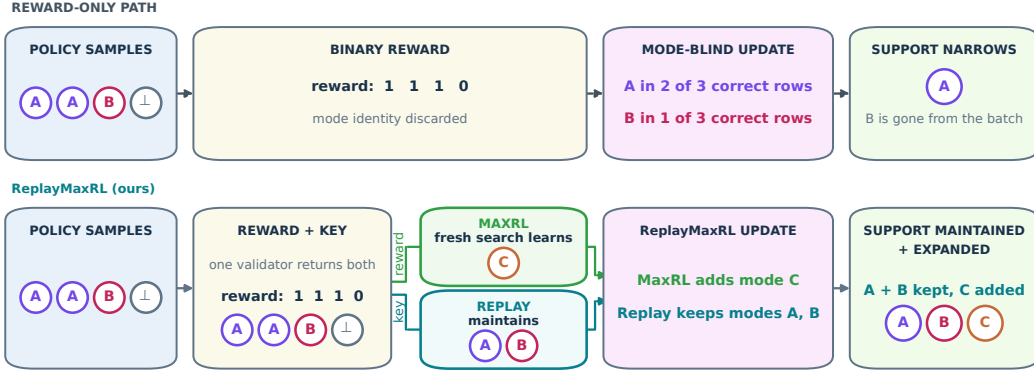


Figure 4: **Fresh learning and memory use the same verified samples.** MaxRL learns from their binary rewards. Replay stores exemplars by canonical key and revisits retained keys with equal weight. In the schematic, fresh samples add C while replay revisits A and B. A stored exemplar can continue to receive a training signal after it becomes rare in fresh samples.

3.1 LEARNING FROM FRESH ROLLOUTS

MaxRL combines success objectives across sampling budgets (Tajwar et al., 2026). Its binary-reward group estimator weights a correct response more strongly when successes are scarce. Comparing replay under both MaxRL and Dr.GRPO (Liu et al., 2025) therefore tests whether memory complements a stronger fresh-learning objective.

Both objectives still learn from what the policy currently samples. A rare mode must be revisited to contribute its own score term; all-correct and all-incorrect groups have zero centered task advantage. Thus even highly successful groups can stop supplying comparative signal. Replay can revisit a banked success without waiting for rediscovery. Appendix F.1 specifies the exact advantages and finite-group interpretation.

3.2 REPLAYING DISTINCT VERIFIED SUCCESSES

Each training prompt owns an initially empty, bounded bank $\mathcal{B}_x$. When a new key is admitted, the bank retains one verified exemplar e_c . Repeated discoveries of that key do not add further entries. A deterministic schedule revisits nonempty prompt banks throughout training. For an exemplar of token length $|e_c|$, the replay loss is

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (3)$$

Each retained key receives the same share of replay, independent of how often it appeared in fresh rollouts. A mode discovered once therefore continues to receive a signal alongside a mode discovered repeatedly. Token normalization sets the loss scale for each exemplar; it does not make exemplar likelihood equal to the total probability of generating its canonical key.

We add this loss to the fresh MaxRL update. Replacing MaxRL with Dr.GRPO yields ReplayDr.GRPO. Replay exemplars stay outside the fresh reward group, so the fresh objective and replay can be varied separately. The controls execute the same bank and scoring code with zero replay gradient; their learned banks and workloads can differ as policies diverge. Banks contain only responses to training prompts. Held-out evaluation therefore tests what training changes in the policy, rather than retrieving answers from the bank. Appendix F gives admission, scheduling, masking, and loss scaling.

3.3 WHY MEMORY ADDRESSES A DIFFERENT PRESSURE

The idealized analysis separates frequency amplification from memory’s persistent signal. Consider an isolated prompt with finitely many categories, at least two correct and one incorrect outcome,

finite initial logits, and $G \geq 2$. With independent canonical-outcome logits, common length normalization, and Euclidean infinitesimal on-policy updates without a competing regularizer, the GRPO, Dr.GRPO, and binary MaxRL mean flows satisfy, after a positive time change,

$$\frac{dq_c}{d\tau} = q_c(q_c - C), \quad \frac{dC}{d\tau} = 2 \text{Var}_{c \sim q}(q_c) \geq 0. \quad (4)$$

A unique initial winner attracts the conditional mass (Theorem C.3). Under the same geometry, bounded fresh-sample scores exert vanishing expected force on a mode as it becomes rare; replay can retain a signal for a banked target without rediscovery (Appendix C.3). The pressure depends on gradient geometry; it does not certify AdamW/PPO or variable-length neural updates.

For the categorical replay surrogate, bank mass P_B and its conditional allocation q_B give

$$-\frac{1}{|\mathcal{B}|} \sum_{c \in \mathcal{B}} \log p_c = -\log P_B + \log |\mathcal{B}| + \text{KL}(U_B \| q_B). \quad (5)$$

Replay rewards banked mass and penalizes imbalance. A fixed bank with constant positive replay dose gives banked modes a probability floor in this flow; unbanked modes may vanish (Theorem C.6). The implemented exemplar loss is not an exact key-probability objective, and neither the floor nor its exemplar counterpart guarantees visibility in eight draws or held-out transfer. Appendix A maps each claim to its basis and scope.

4 EXPERIMENTAL DESIGN

We study `pass@8` and `distinct@8` through matched local training and frontier model evaluation, separating learning effects from the behavior of fixed deployments.

Local training experiments. We compare Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B across all five Level-1 domains, with eight training passes and five paired seeds per comparison except where marked in the figures. Matched experiments add replay to Dr.GRPO and cross Dr.GRPO/MaxRL with replay off/on at all three scales. We repeat the factorial on all five Level-2 domains with Qwen2.5-0.5B. Evaluation averages four eight-sample draws on 128 held-out prompts; intervals are nominal pointwise 95% intervals over paired seeds. Controls include frequency-weighted replay, UCPO, RLEP, and fixed Semantic-MaxEnt. Appendix Table 6 gives their comparison scope, and Appendix D.5 specifies training and evaluation budgets.

Frontier model experiments. We evaluate GPT-5.6 Sol, GPT-5.4, Grok 4.3, Kimi K3, and Claude Opus 4.8 with explicit reasoning disabled across all five domains and Levels 1–3. Each produces eight stateless responses per prompt on 32 fixed prompts per domain and level, totaling 480 prompts per model. Matched medium-reasoning responses preserve prompt wording and 8,192-token output limits (Appendix J.5). Main scores use a frozen formatting normalizer, equal domain weights, and all prompts, including unsolved ones. Appendix J retains the earlier seven-model medium cohort (128 prompts per domain and level), temperature controls, and cross-model mixtures. These exploratory inference-only comparisons characterize deployment behavior; they do not apply replay or identify training or model-size effects.

5 RESULTS

Training can replace varied attempts with one repeated answer. On Python, all five Qwen2.5-0.5B Dr.GRPO seeds begin with almost eight unique raw responses per eight-sample draw. By steps 288–480, every saved draw repeats one byte-identical response per prompt, matching greedy on all 128 test prompts; this persists through training. Only 1.5% of the control’s training groups contain both correct and incorrect responses, compared with 18.5% under replay. Thus most control updates carry no fresh task gradient, while verified memory supplies a separate learning signal (Appendix H.3). This is observed output collapse under the tested sparse-reward recipe, including incorrect repeated answers.

Concentration among *correct* outputs is separately measurable. On Graph and PantryPlan, all twelve Dr.GRPO/GRPO before–after comparisons increase correct-key collision; ReplayDr.GRPO

lowers it in all six matched comparisons (Appendix Figure 20). These retrospective contrasts describe prompts with enough correct samples in both conditions, with sampling sensitivity reported in Appendix H.1; the raw-breadth precheck is in Appendix H.2. Replay can lower per-sample correctness on that selected population while improving full-test pass@8 . The following experiments therefore report success and verified breadth jointly.

5.1 EXPERIMENT 1: REPLAY IMPROVES SUCCESS AND SAMPLED SUPPORT

Verified memory improves both outcomes across scales. ReplayDr.GRPO raises mean pass@8 and distinct@8 in every tested model-domain comparison (Figure 5A). Raw verified mode count also increases in every paired training seed. The effect spans valid configurations, computation routes, and tested program behaviors.

A ReplayDr.GRPO (ours) – Dr.GRPO across three model scales

Δ pass@8							Δ distinct@8						
Qwen 0.5B	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	Qwen 0.5B	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
Falcon 1B	+0.15	+0.31 [†]	+0.98	+0.64	+0.12	+0.44 [†]	Falcon 1B	+0.68	+0.42 [†]	+1.11	+0.66	+0.94	+0.77 [†]
Qwen 3B	+0.15	+0.14	+0.59	+0.41	+0.11	+0.28	Qwen 3B	+0.58	+0.33	+0.78	+0.43	+0.67	+0.56
	Graph	Countdown	Python	MathIR	Pantry	Average		Graph	Countdown	Python	MathIR	Pantry	Average

B Initial reference + alternatives – terminal Dr.GRPO at Qwen2.5-0.5B

Δ pass@8							Δ distinct@8						
Before training	+0.13	-0.38	-0.17	-0.29	+0.38	-0.07	Before training	+0.24	-0.38	-0.17	-0.29	+1.90	+0.26
ReplayDr.GRPO (ours)	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	ReplayDr.GRPO (ours)	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
MaxRL (no replay)	+0.21	+0.10	+0.00	-0.07	+0.01	+0.05	MaxRL (no replay)	+0.35	+0.13	+0.00	-0.07	+0.01	+0.09
UCPO	-0.00	+0.13	+0.00	+0.03	+0.05	+0.04	UCPO	-0.00	+0.18	+0.00	+0.03	+0.05	+0.05
RLEP-Dr	+0.10	+0.10	+0.00	+0.13	+0.00	+0.07	RLEP-Dr	+0.15	+0.13	+0.00	+0.12	+0.00	+0.08
Semantic-MaxEnt	+0.00	+0.14	+0.00	-0.12	+0.20	+0.04	Semantic-MaxEnt	+0.00	+0.18	+0.00	-0.12	+1.12	+0.24
GRPO	+0.04	+0.16	+0.00	-0.05	+0.02	+0.03	GRPO	+0.05	+0.17	+0.00	-0.05	+0.02	+0.04
	Graph	Countdown	Python	MathIR	Pantry	Average		Graph	Countdown	Python	MathIR	Pantry	Average

Figure 5: Canonical replay improves correctness and verified breadth. (A) ReplayDr.GRPO minus Dr.GRPO at all three scales. (B) Qwen2.5-0.5B alternatives minus Dr.GRPO. Columns measure probability points and expected verified-key counts, respectively. The green Average weights domains equally within seed. A dagger marks four paired seeds in Falcon Countdown and its Average; other cells use five. Intervals are nominal 95% intervals for five-seed comparisons. Purple denotes ReplayDr.GRPO; bold entries mark column maxima in Panel B.

At Qwen2.5-0.5B, Countdown pass@8 rises from .480 to .672, while sampled verified computation routes more than triple, from .487 to 1.637. The gain beyond the first correct outcome is +.959 modes (95% interval $[+.863, +1.054]$). Python illustrates a different outcome: correctness rises from .172 to .528, but its extra-mode gain remains uncertain. More successful samples need not mean a more balanced distribution over correct outputs (Appendix G.2).

Balancing keys contributes beyond reusing successes. ReplayDr.GRPO has the largest mean correctness gain among the direct Qwen2.5-0.5B alternatives in every domain (Figure 5B). A closer ablation changes only the replay target from equal key weights to fresh discovery-frequency weights. Uniform weighting adds .318 extra modes per prompt on the five-domain average ($[+.272, +.358]$), driven by Graph, Countdown, and PantryPlan. Its aggregate correctness effect is inconclusive, as are the Python and MathIR breadth contrasts. This supports key balancing as a contributor to breadth within the tested tasks (Appendix G.6, Figure 19).

5.2 EXPERIMENT 2: REPLAY COMPLEMENTS A STRONGER FRESH OBJECTIVE

Memory adds value when fresh learning already uses MaxRL. The paired comparisons in Figure 6 separate replay from the choice of fresh objective. At all three scales, replay improves domain-averaged correctness and verified mode count under both Dr.GRPO and MaxRL. Its ben-

efit therefore persists beyond MaxRL’s sampling-aware fresh objective; replay and stronger on-policy learning address complementary parts of the training process.

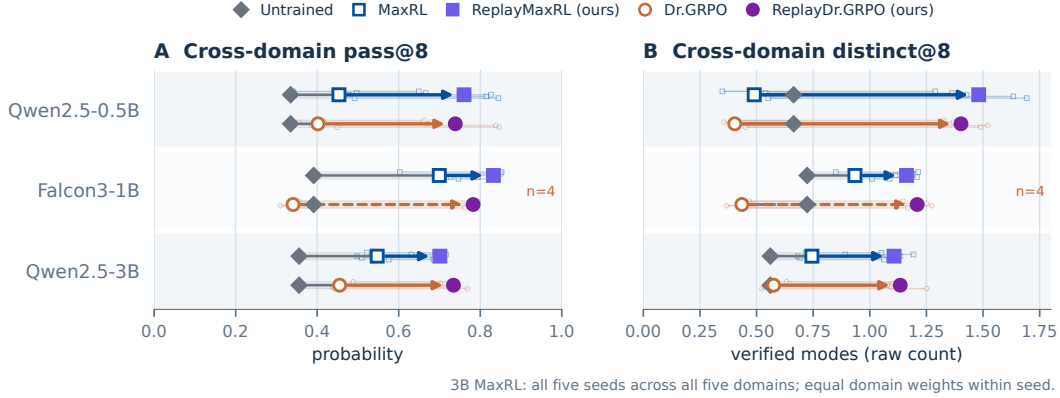


Figure 6: **Replay complements both fresh objectives at multiple scales.** Each row shows Dr.GRPO/ReplayDr.GRPO and MaxRL/ReplayMaxRL on both endpoints. Diamonds are initial models; open/filled markers are training without/with replay. Tracks average domains within seed; faint paths show seeds. All MaxRL tracks use five paired seeds in all five domains, including 3B. Falcon Dr.GRPO uses four common seeds; the other Dr.GRPO tracks use five. Appendix Figure 11 reports each domain’s correctness and verified-mode results at every scale.

ReplayMaxRL’s domain-averaged pass@8 gains are 30.7, 13.3, and 15.4 percentage points at 0.5B, 1B, and 3B, respectively. At 3B, correctness rises from .547 to .701 and verified modes from .744 to 1.107. These averages retain domain differences: Falcon Graph has a negative, inconclusive raw-support effect, and the 3B Graph and Python correctness intervals include zero. The comparison identifies the benefit of the replay update, not which stored exemplars cause held-out gains or a superadditive interaction with MaxRL. Appendix Figure 10 reports domain effects; Appendices G.2 and G.4 give aggregate estimates and trajectories throughout training.

5.3 EXPERIMENT 3: REPLAY WITHIN A SECOND TASK CONSTRUCTION

The benefit extends to harder task constructions. At Qwen2.5-0.5B, replay improves average correctness and sampled support under both fresh objectives within both levels (Figure 7). Each comparison holds the level’s prompts and fresh objective fixed. Separate test populations and protocol differences prevent interpreting cross-level changes as the isolated effect of difficulty.

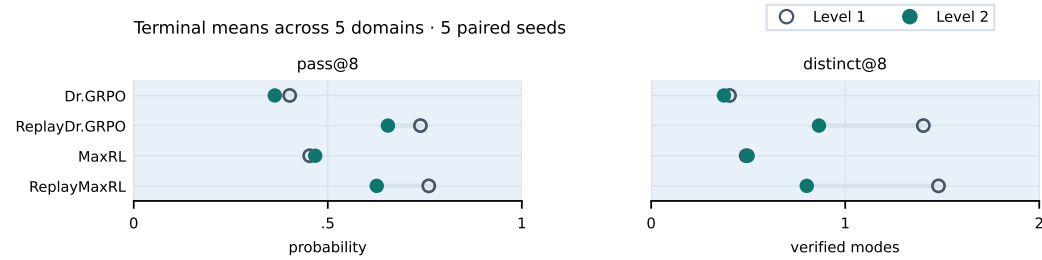


Figure 7: **Replay across difficulty levels at Qwen2.5-0.5B.** Open/filled markers denote Level 1/2. Terminal means average all five domains equally within each of five paired seeds. Both levels and all four methods share these seeds. The benchmark construction is described in Section 2.2; different test populations and parts of the protocol prevent an isolated difficulty interpretation (Appendix G.3).

Level-2 Graph gives the clearest example. Dr.GRPO and MaxRL reach pass@8 of .202 and .434, versus .762 and .739 with replay; the weakest replay seed exceeds the strongest non-replay seed. PantryPlan also improves on both primary outcomes, adding .186 and .175 extra modes under Dr.GRPO and MaxRL. The other domains’ correctness and raw-support intervals include zero (Appendix G.3). Pantry’s collision nevertheless increases on sparse jointly correct subsets: improved sampled breadth does not guarantee lower conditional concentration.

Guidance, mode definitions, and sample budget test what breadth means. Removing strategy hints lowers Python correctness, although replay retains success where Dr.GRPO fails on the neutral-wording probes (Appendix M). Merging opposite factor-pair members removes the original-wording Level-3 replay breadth gain (.419 $\rightarrow$ 0), while neutral wording retains a .500 gain; neither key identifies algorithms (Appendix L.3). Larger sampling budgets distinguish concentration from alternatives missed at eight draws. Most trained MathIR conditions remain concentrated at 64 draws. Removing MathIR guidance broadens some hosted GPT outputs even at a fixed number of correct draws (Appendix N.2), while Grok’s neutral Level-3 Pantry breadth rises from 3.97 to 8.44 modes with $\text{pass}@K$ fixed at 1.0. These exploratory budget probes use sixteen problems per cell and measure sampled alternatives, not exhaustive coverage (Appendix N).

Saved alternatives can support adaptation to changed requirements. On 32 held-out Pantry problems and two fixed seed pairs, replay improves survival of eight saved plans under independently chosen feasible outages by 23.83 percentage points and reduces recovery by 2.08 calls. Gains also appear with tuned-temperature and diversity-prompt controls (Appendix L.1). Original correctness improves too, so this establishes better adaptation under the tested intervention without isolating diversity as its cause.

5.4 HOSTED MODELS: CORRECTNESS DOES NOT DETERMINE BREADTH

Verified concentration also appears in hosted deployments. In a selected illustration from the medium-reasoning cohort, GPT-5.6 Sol answers all 1,024 Level-3 Graph draws correctly, yet eight draws contain only 1.16 distinct verified colorings on average. Correct-pair collision is 95.1%, compared with a 19.0% uniform-key reference. Thus narrow breadth can persist even when every response succeeds (Appendix I).

More samples need not recover the available modes. At 512 draws across all five domains and Levels 1–3 (Figure 8), Graph averages 1.72–2.63 modes against 6.03–6.97 available. Doubling from 256 to 512 draws adds at most 0.074 mean modes in Python Levels 2–3; MathIR Levels 2–3 recover at most one mode per problem. Other cells, especially Pantry and Level-1 Python, continue to gain modes with additional sampling.

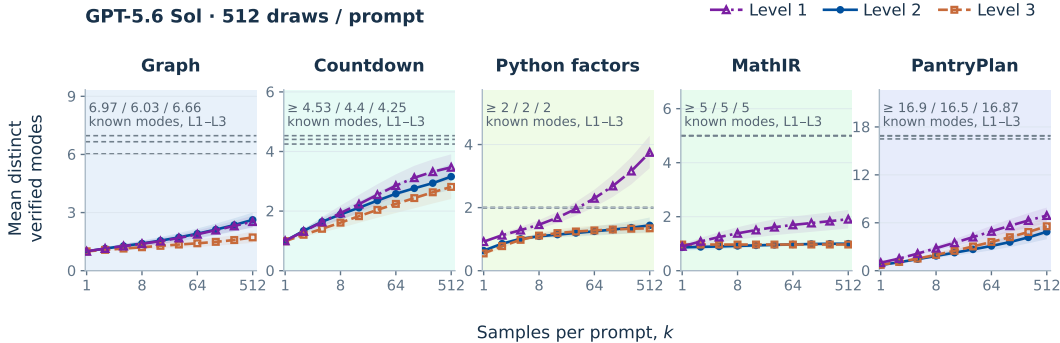


Figure 8: **More samples need not recover the available modes.** GPT-5.6 Sol, medium reasoning, original wording; thirty-two problems per cell, Levels 1–3, and 512 responses per problem. Curves average distinct verified modes in random k -subsets, including failures; bands are pointwise 95% problem-bootstrap intervals. Gray lines show mean known support: exact for Graph and certified lower bounds elsewhere. Frozen formatting normalization; Appendix N.3 gives strict results.

Table 1 summarizes the separate reasoning-disabled evaluation across all domains and levels. On the same 480 prompts, disabling reasoning lowers overall $\text{pass}@8$ and $\text{distinct}@8$ for all five models versus medium reasoning. Falling correctness prevents this comparison from isolating conditional concentration (Appendix J.5). Appendix J and Figure 25 give the seven-model medium results. These deployment observations identify no training cause: hosted collision pools correct pairs; training collision weights eligible prompts equally.

Model	Level 1		Level 2		Level 3	
	pass@8 (%)	# modes	pass@8 (%)	# modes	pass@8 (%)	# modes
 GPT-5.6 Sol	82.5	1.31	66.9	0.76	76.2	0.88
 GPT-5.4	82.5	1.67	71.2	0.92	70.0	0.94
 Grok 4.3	82.5	1.64	86.9	1.41	90.0	1.45
 Kimi K3	92.5	1.91	92.5	1.37	95.6	1.59
 Claude Opus 4.8	91.2	1.46	100.0	1.62	100.0	1.51

Table 1: **Hosted success and output diversity with reasoning disabled.** Each level averages five domains equally: 32 prompts per domain and eight responses per prompt. `pass@8` is the fraction with any verified response; # modes is mean `distinct@8`, including failures. The frozen formatting normalizer applies throughout. Appendix J.5 gives matched medium-reasoning results.

Different models also expose complementary alternatives. Nine of the 21 four-plus-four response mixtures have positive pointwise 95% intervals in verified breadth against both eight-response constituents. Gains over the average constituent persist at matched successful-response counts (Appendix L.2). This comparison fixes response counts; token costs can differ between models.

Temperature provides another way to vary the sampled alternatives (Figure 24). On 480 matched prompts with GPT-5.6 Sol and reasoning `none`, raising T from 0 to 1.5 increases normalized empirical `pass@8` from 65.6% to 79.8% and mean `distinct@8` from .673 to 1.125. At $T = 2$, these are 79.4% and 1.146; paired 95% intervals for $T = 2$ minus $T = 1.5$ include zero on both axes. Grok shows no clear breadth gain at the tested higher temperature; Kimi suffers severe truncation (Appendix J.2).

6 RELATED WORK

RLHF and reasoning RL can reduce output diversity, entropy and high-budget coverage (Kirk et al., 2024; Dang et al., 2025; Cui et al., 2025; Yue et al., 2025). MODEBENCH measures verified output identities; semantic clustering instead infers equivalence from language (Farquhar et al., 2024). Sampling-based selection motivates preserving proposal alternatives (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Lightman et al., 2024); our Pantry follow-up measures adaptation to changed constraints, while general search utility remains untested.

Outcome bonuses, set prediction, distribution matching and uniform-correct objectives also encourage broader support (Song et al., 2025; Chen et al., 2025; Li et al., 2025; 2026a;b; Lochab et al., 2026). MaxRL combines objectives across sampling budgets (Tajwar et al., 2026); we ask whether memory adds value. Replay can mitigate forgetting (Lin, 1992; Rolnick et al., 2019), and RLEP reuses successful reasoning trajectories (Zhang et al., 2025). Our memory balances execution keys independently of discovery frequency, connecting replay to behavioral archives in quality-diversity methods (Mouret & Clune, 2015; Ecoffet et al., 2021).

7 CONCLUSION

Reliable answering and access to verified alternatives are different properties. MODEBENCH makes both observable, and ReplayMaxRL uses the same execution identities to keep discovered successes involved in learning. Controlled comparisons show that memory complements stronger fresh optimization; key weighting, prompt interventions, larger sampling budgets, and hosted mixtures reveal what determines the alternatives observed.

Replay depends on discovery and bank capacity. A fixed-bank audit finds improving central exemplar scores alongside a declining tail (Appendix H.4); it does not establish retention of every key. Matched retention tests and portfolios with equal numbers of correct candidates would clarify what memory preserves and when alternatives add value, connecting the idealized mechanism to neural behavior and adaptation.